

Theoretical problems in concurrency

Dining Philosophers Problem

- No two philosophers can use the same fork at the same time (mutual exclusion)
- No philosopher must starve to death (avoid deadlock and starvation)

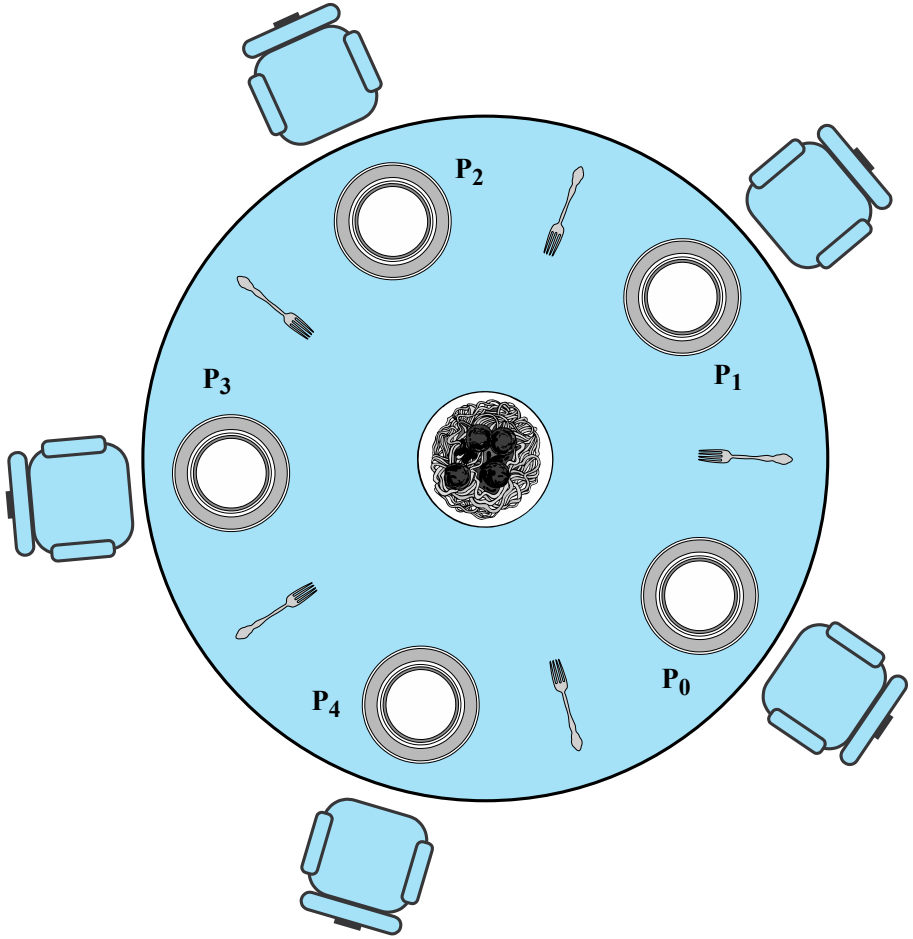


Figure 6.11 Dining Arrangement for Philosophers

```
/* program      diningphilosophers */
semaphore fork [5] = {1};
int i;
void philosopher (int i)
{
     while (true) {
          think();
          wait (fork[i]);
          wait (fork [(i+1) mod 5]);
          eat();
          signal(fork [(i+1) mod 5]);
          signal(fork[i]);
     }
}
void main()
{
     parbegin (philosopher (0), philosopher (1), philosopher
(2),
          philosopher (3), philosopher (4));
     }
```

What can you do?

- You can add five additional forks.
- You can add a room attendant who only allows 4 philosophers to enter the room at a given time.
  - Then at least one philosopher will have access to two forks.

```
/* program diningphilosophers */
semaphore fork[5] = {1};
semaphore room = {4};
int i;
void philosopher (int i)
{
    while (true) {
     think();
     wait (room);
     wait (fork[i]);
     wait (fork [(i+1) mod 5]);
     eat();
     signal (fork [(i+1) mod 5]);
     signal (fork[i]);
     signal (room);
    }

}
void main()
{
    parbegin (philosopher (0), philosopher (1), philosopher (2),
          philosopher (3), philosopher (4));
}
```

- A solution to the Dining Philosophers using Monitors

```
monitor dining_controller;
cond ForkReady[5];          /* condition variable for synchronization */
boolean fork[5] = {true};       /* availability status of each fork */

void get_forks(int pid)          /* pid is the philosopher id number */
{
   int left = pid;
   int right = (++pid) % 5;
   /*grant the left fork*/
   if (!fork[left])
      cwait(ForkReady[left]);         /* queue on condition variable */
   fork[left] = false;
   /*grant the right fork*/
   if (!fork[right])
      cwait(ForkReady[right]);        /* queue on condition variable */
   fork[right] = false:
}
void release_forks(int pid)
{
   int left = pid;
   int right = (++pid) % 5;
   /*release the left fork*/
   if (empty(ForkReady[left])      /*no one is waiting for this fork */
      fork[left] = true;
   else                        /* awaken a process waiting on this fork */
      csignal(ForkReady[left]);
   /*release the right fork*/
   if (empty(ForkReady[right])     /*no one is waiting for this fork */
      fork[right] = true;
   else                        /* awaken a process waiting on this fork */
      csignal(ForkReady[right]);
}
```

```
void philosopher[k=0 to 4]           /* the five philosopher clients */
{
   while (true) {
      <think>;
      get_forks(k);          /* client requests two forks via monitor */
      <eat spaghetti>;
      release_forks(k);      /* client releases forks via the monitor */
   }
}
```

Producer/Consumer Problem

General Statement:	One or more producers are generating data and placing these in a buffer
	A single consumer is taking items out of the buffer one at a time
	Only one producer or consumer may access the buffer at any one time
The Problem:	Ensure that the producer won't try to add data into the buffer if its full, and that the consumer won't try to remove data from an empty buffer

Initial solution

- Use two binary semaphores.
  - s to make sure only one can enter the buffer.
  - $delay$ to make sure that consumer has to wait when there are no elements to consume.
- Is this, okay?

```
/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;
void producer()
{
     while (true) {
          produce();
          semWaitB(s);
          append();
          n++;
          if (n==1) semSignalB(delay);
          semSignalB(s);
     }
}
void consumer()
{
     semWaitB(delay);
     while (true) {
          semWaitB(s);
          take();
          n--;
          semSignalB(s);
          consume();
          if (n==0) semWaitB(delay);
     }
}
void main()
{
     n = 0;
     parbegin (producer, consumer);
}
```

Figure 5.12 An Incorrect Solution to the Infinite-Buffer Producer/Consumer Problem Using Binary Semaphores

	Producer	Consumer	s	n	Delay
1			1	0	0
2	`semWaitB(s)`		0	0	0
3	`n++`		0	1	0
4	`if (n==1) (semSignalB(delay))`		0	1	1
5	`semSignalB(s)`		1	1	1
6		`semWaitB(delay)`	1	1	0
7		`semWaitB(s)`	0	1	0
8		`n--`	0	0	0
9		`semSignalB(s)`	1	0	0
10	`semWaitB(s)`		0	0	0
11	`n++`		0	1	0
12	`if (n==1) (semSignalB(delay))`		0	1	1
13	`semSignalB(s)`		1	1	1
14		`if (n==0) (semWaitB(delay))`	1	1	1
15		`semWaitB(s)`	0	1	1
16		`n--`	0	0	1
17		`semSignalB(s)`	1	0	1
18		`if (n==0) (semWaitB(delay))`	1	0	0
19		`semWaitB(s)`	0	0	0
20		`n--`	0	–1	0
21		`semSignalB(s)`	1	–1	0

```
/* program producerconsumer */
int n;
binary_semaphore s = 1, delay = 0;
void producer()
{
     while (true) {
          produce();
          semWaitB(s);
          append();
          n++;
          if (n==1) semSignalB(delay);
          semSignalB(s);
     }
}
void consumer()
{
     int m; /* a local variable */
     semWaitB(delay);
     while (true)  {
          semWaitB(s);
          take();
          n--;
          m = n;
          semSignalB(s);
          consume();
          if (m==0) semWaitB(delay);
     }
}
void main()
{
     n = 0;
     parbegin (producer, consumer);
}
```

Figure 5.13 A Correct Solution to the Infinite-Buffer Producer/Consumer Problem Using Binary Semaphores

```
/* program  producerconsumer */
semaphore n = 0, s = 1;
void producer()
{
     while (true) {
          produce();
          semWait(s);
          append();
          semSignal(s);
          semSignal(n);
     }
}
void consumer()
{
     while (true) {
          semWait(n);
          semWait(s);
          take();
          semSignal(s);
          consume();
     }
}
void main()
{
     parbegin (producer, consumer);
}
```

Figure 5.14 A Solution to the Infinite-Buffer Producer/Consumer Problem Using Semaphores

Questions

- What if we swap the $semSignal(s)$ and $semSignal(n)$?
  - It would not affect the correctness.
- What if we swap the $semWait(n)$ and $semWait(s)$?
  - This is fatal.
  - There is a chance that consumer can get hold of the buffer when the buffer is empty, then it would not release the buffer therefore producer can not add to the buffer. This would be a deadlock.

```
/* program boundedbuffer */
const int sizeofbuffer = /* buffer size */;
semaphore s = 1, n= 0, e= sizeofbuffer;
void producer()
{
     while (true) {
          produce();
          semWait(e);
          semWait(s);
          append();
          semSignal(s);
          semSignal(n);
     }
}
void consumer()
{
     while (true) {
          semWait(n);
          semWait(s);
          take();
          semSignal(s);
          semSignal(e);
          consume();
     }
}
void main()
{
     parbegin (producer, consumer);
}
```

Figure 5.16 A Solution to the Bounded-Buffer Producer/Consumer Problem Using Semaphores

Readers/Writers Problem

- A data area is shared among many processes
    - Some processes only read the data area, (readers) and some only write to the data area (writers)
- Conditions that must be satisfied:
    - Any number of readers may simultaneously read the file
    - Only one writer at a time may write to the file
    - If a writer is writing to the file, no reader may read it

Solution 1—refer the handout.

- Readers have priority.
- Writer's procedure is simple.
  - Use the semaphore $wsem$ to ensure mutual exclusion to the common data area.
- For readers,
  - First, need to get hold of semaphore x to update the $readcount$ variable.
    - The purpose of the semaphore x is to ensure the mutual exclusion to the $readcount$ variable.
  - The first reader, captures the $wsem$ semaphore and prevent writers from writing anything to the common data area.
  - The last reader who finishes reading sends a signal (gives up) the semaphore.

Algorithm 1 Readers have priority

```
1:  readercount ← 0
2:  Semaphore read_lock = 1                ▷ Used for updating the readcount.
3:  Semaphore wsem = 1 ▷ Used to make sure that only one writer can write at a given time and make sure that
    weiter cannot interfere with reader.
4:  function READER( )
5:      while true do
6:          semWait(read_lock)             ▷ Locking the readcount before updating.
7:          readcount ← readcount + 1
8:          if readcount == 1 then
9:              semWait(wsem)              ▷ Make sure that no writer can enter while the current reader is reading.
10:         semSignal(read_lock)
11:         READUNIT()
12:         semWait(read_lock)
13:         readcount ← readcount − 1
14:         if readcount == 0 then         ▷ No reader is reading.
15:             semSignal(wsem)            ▷ Writers can now access common read area.
16:         semSignal(read_lock)
17: function WRITER( )
18:     while true do
19:         semWait(wsem)                  ▷ Requests the common area.
20:         WRITEUNIT()
21:         semSignal(wsem)                ▷ Leaves the common area.
22: function MAIN( )
23:     readcount ← 0
24:     parbegin(reader, writer)
```

Implications?

- If there is a constant flow of readers to the system, writers may never access the common data area.

Solution 2—Refer the handout

- Writer’s have priority.
- Idea is that whenever some writer declares the desire to write, **no new reader** is guaranteed to access the system until at least one of the writers write something to the common data area.

Algorithm 2 Writers have priority

```
1: readercount ← 0, writecount ← 0
2: Mutex r_mutex, w_mutex                          ▷ Used for updating readcount and writecount
3: Semaphore readblock = 1, writeblock = 1, writepending = 1
4: function READER( )
5:     while true do
6:         semWait(writepending)
7:         semWait(readblock) ▷ If a reader is blocked at readerblock then all the other subsequent readers are
           blocked at writepending semaphore
8:         r_mutex.lock()
9:         readcount ← readcount + 1
10:        if readcount == 1 then
11:            SemWait(writeblock)                  ▷ First reader blocks writers for other readers.
12:        r_mutex.unlock()
13:        semSignal(readblock)                     ▷ releases readblock so that other readers or writers can use it.
14:        semSignal(writepending)                  ▷ writepending = 1 again.
15:        READUNIT()
16:        r_mutex.lock()
17:        readcount ← readcount − 1
18:        if readcount == 0 then
19:            semSignal(writeblock)                ▷ If no other readers are available then writeblock is released.
20:        r_mutex.unlock()
21: function WRITER( )
22:     while true do
23:         w_mutex.lock()
24:         writecount ← writecount + 1
25:         if writecount == 1 then
26:             semWait(readblock)
27:         w_mutex.unlock()
28:         semWait(writeblock)
29:         WRITEUNIT()
30:         semSigmal(writeblock)
31:         w_mutex.lock()
32:         writecount ← writecount − 1
33:         if writecount == 0 then
34:             semSignal(readblock)
35:         w_mutex.unlock()
36: function MAIN( )
37:     readcount ← 0, writecount ← 0
38:     parbegin(reader, writer)
```

Readers only in the system	•*wsem* set •no queues
Writers only in the system	•*wsem* and *rsem* set •writers queue on *wsem*
Both readers and writers with read first	•*wsem* set by reader •*rsem* set by writer •all writers queue on *wsem* •one reader queues on *rsem* •other readers queue on *z*
Both readers and writers with write first	•*wsem* set by writer •*rsem* set by writer •writers queue on *wsem* •one reader queues on *rsem* •other readers queue on *z*

Implications

- Readers could starve.
- There is a solution for this problem where neither starves.
  - You can use a queue using a semaphore.
  - Think about this idea.